Competitive Intelligence Study

Using AI to Surface User Sentiment Across Competing Products

Meghna Bhagwat |Product Manager | LinkedIn | Try the tool

Contents

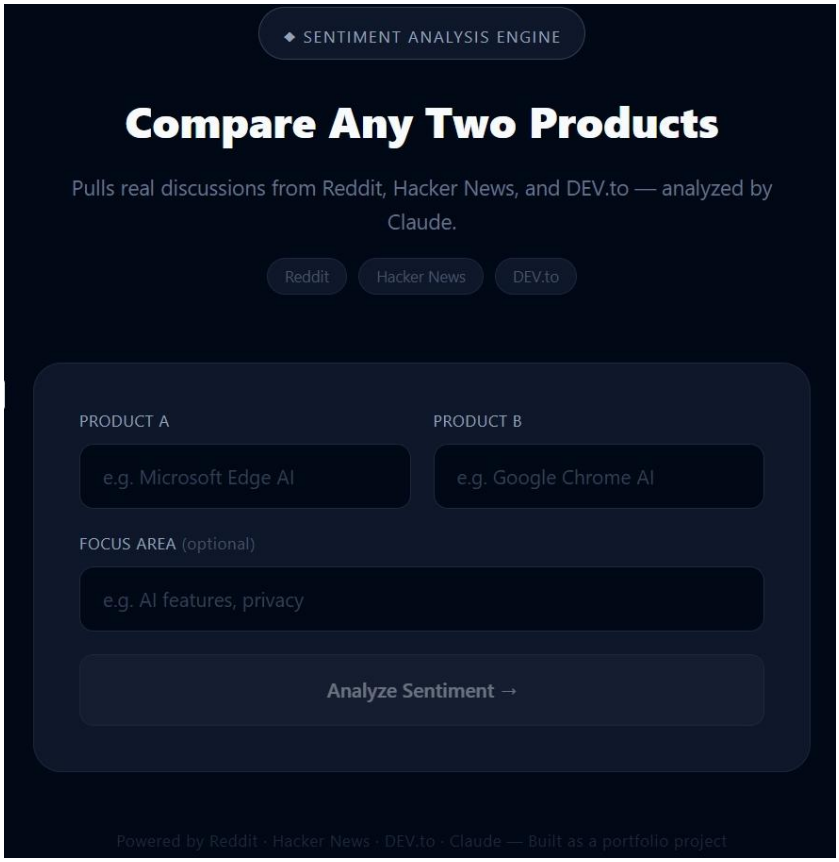
1. What I built
2. Analysis – Microsoft Edge AI vs Google Chrome AI
3. Analysis – Microsoft Teams vs Slack
4. Keeping the analysis grounded
    a. Why the model only summarizes
    b. How I kept it grounded in the posts
    c. What it still gets wrong
    d. Telling a widespread signal from a loud anecdote
5. How I tested it

What I built

A browser-based sentiment analysis engine that pulls real user discussions from Reddit, Hacker News, and DEV.to, then uses Claude to compare two competing products side by side. No surveys. Just what users say, unprompted.

I applied it to two competitive pairs in Microsoft's ecosystem to identify unmet user needs and surface product opportunities grounded in real data.

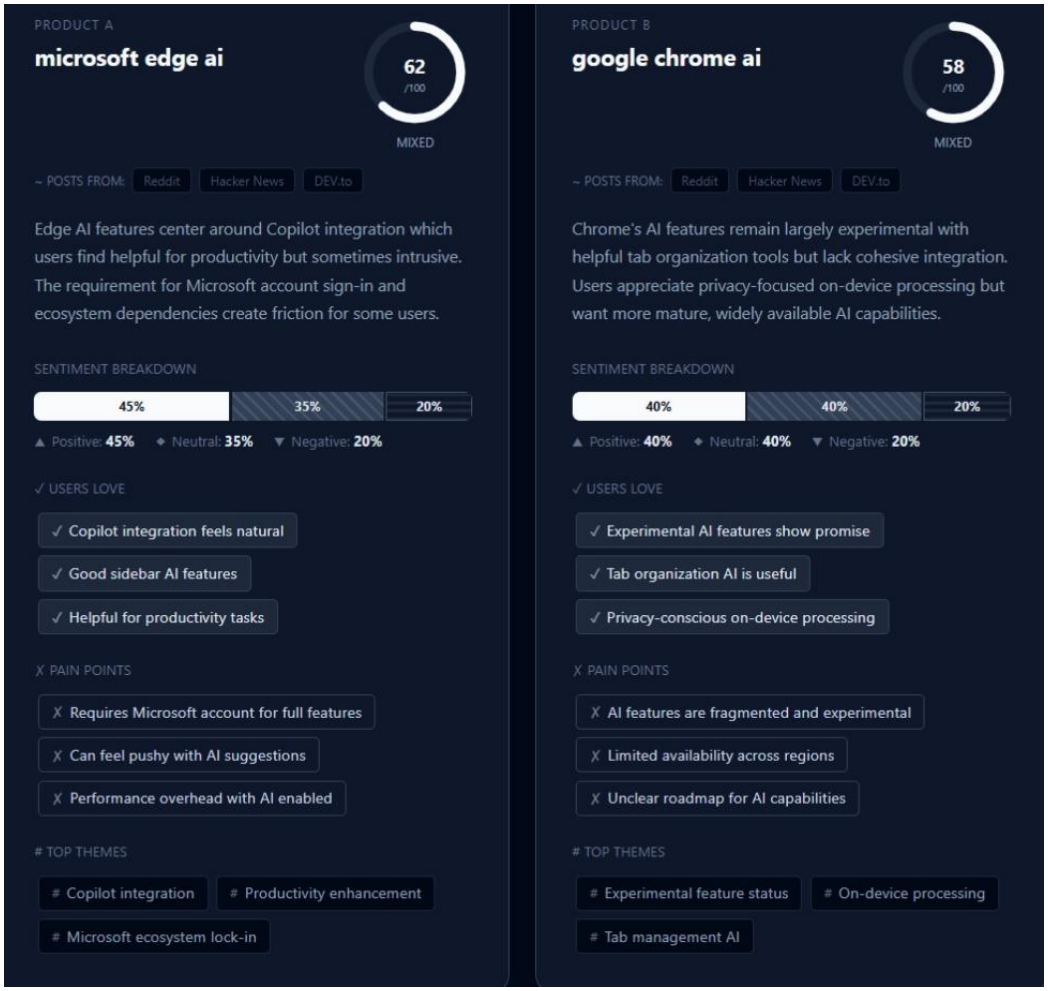
Tools: Reddit API, Hacker News (Algolia), DEV.to API, Claude Haiku 4.5, Vercel serverless functions.

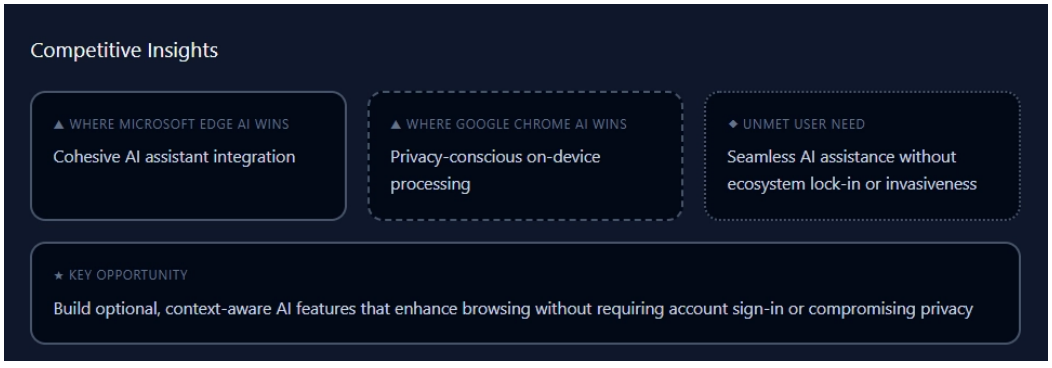


Microsoft Edge AI vs Google Chrome AI

Focus: AI features and browser-integrated intelligence.

Microsoft Edge AI	Google Chrome AI
Sentiment Score: 62/100	Sentiment Score: 58/100
45% Positive · 35% Neutral · 20% Negative	40% Positive · 40% Neutral · 20% Negative





Where Edge AI Wins	Cohesive AI assistant integration - This integration feels like a natural extension of the browser.

Where Chrome AI Wins	Privacy-conscious on-device processing - Users trust that their data stays local.
Key Opportunity	Build optional, context-aware AI features that enhance browsing without requiring account sign-in or compromising privacy.

My Recommendation

The data reveals a tension neither product has resolved: users want rich AI features and privacy. Right now, they have to choose one or the other.

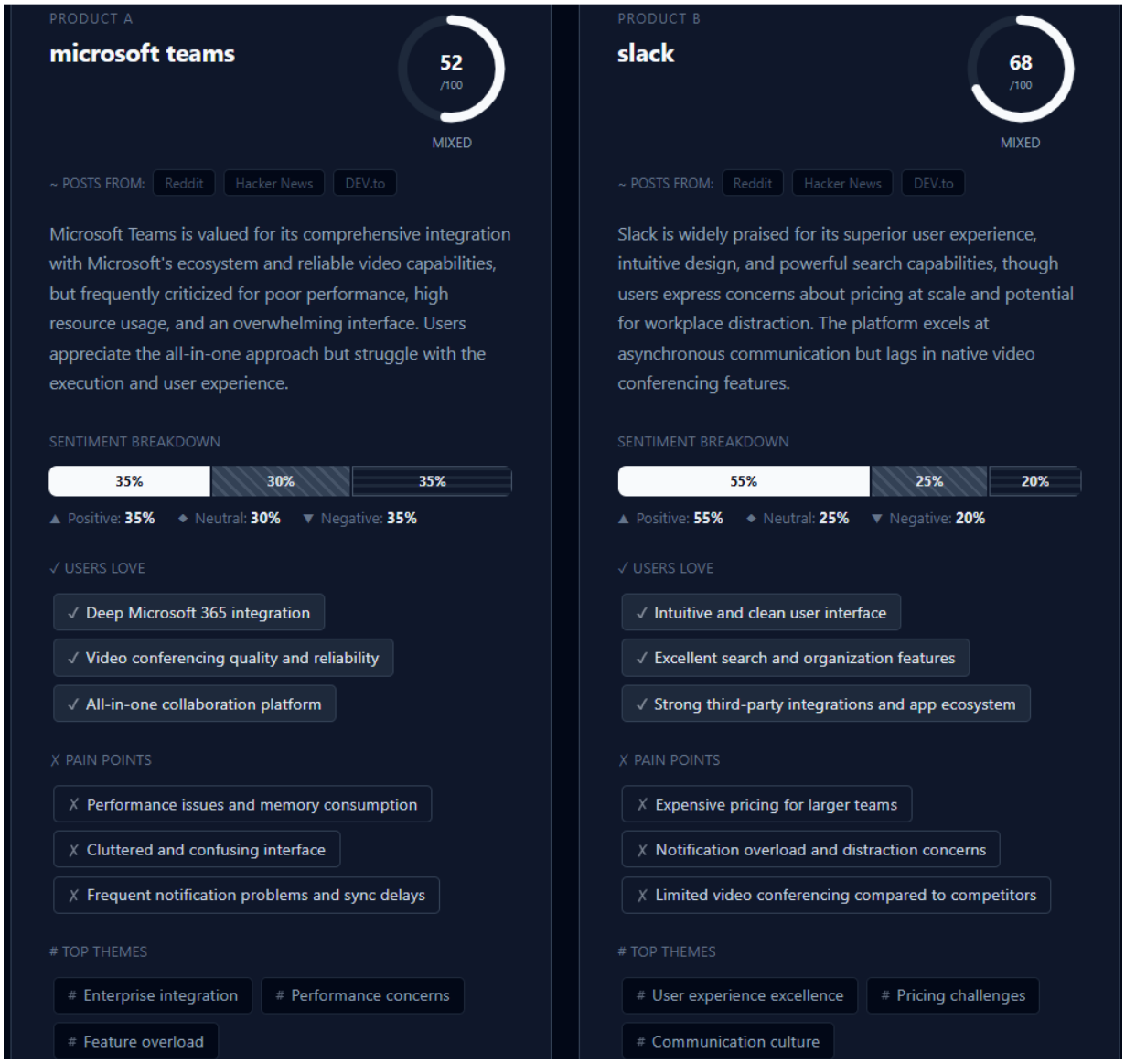
One of the underserved users here is a graduate student, someone who needs AI-powered research assistance in the browser but won't trade their privacy to get it. Edge AI gives them features but takes their data. Chrome AI respects their privacy but offers minimal AI capability. Neither wins this user.

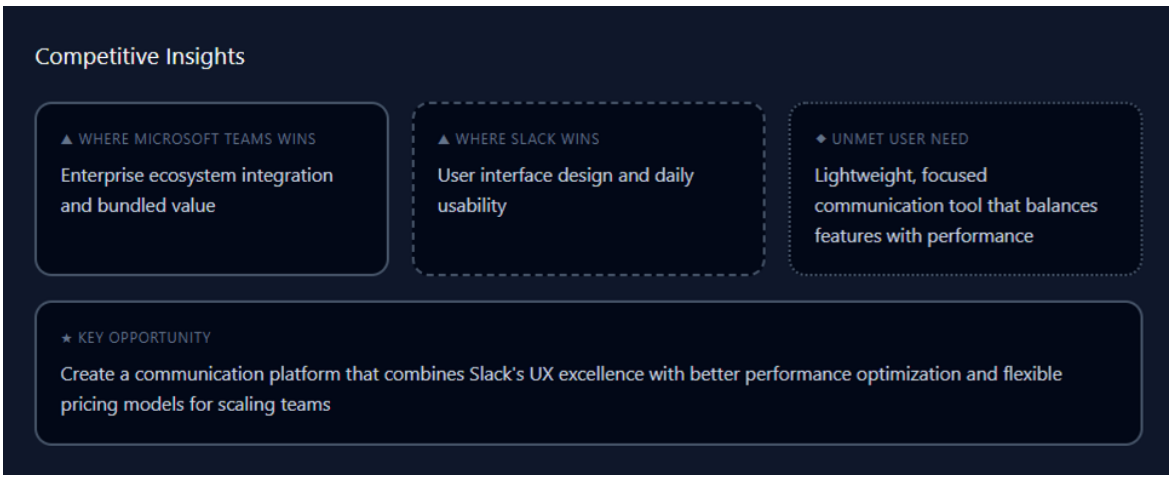
Google is better positioned to close this gap. An AI research assistant that synthesizes credible academic sources directly in the browser, with no login required, would own the student user through their academic years — and create a natural pipeline into Google's professional tools beyond graduation.

Microsoft Teams vs Slack

Focus: Team collaboration, communication, and developer experience

Microsoft Teams	Slack
Sentiment Score: 52/100	Sentiment Score: 68/100
35% Positive · 30% Neutral · 35% Negative	55% Positive · 25% Neutral · 20% Negative





Where Teams Wins	Enterprise ecosystem integration - Deep M365 integration and reliable video conferencing anchor it in large organizations.
Where Slack Wins	Interface design and daily usability - Users consistently define it as intuitive, clean and pleasant to work in.

Key Opportunity	A collaboration tool that combines Slack's UX excellence with better performance optimization and flexible pricing for scaling teams.

My Recommendation

Both products have made tradeoffs that leave the same user stranded: the developer. Teams traded UX simplicity for enterprise capability. Slack traded video reliability for UX excellence. Neither has optimized for developer hardware constraints. Both are memory-heavy in an environment where every resource counts.

The opportunity for Microsoft is a "Developer Mode", which is a stripped-down, resource-efficient version of Teams with simplified navigation, powerful search, and reliable video, without the enterprise clutter developers never use. This directly addresses Microsoft's vulnerability of losing small and mid-size tech companies to Slack, while strengthening ecosystem lock-in from startup to enterprise.

Keeping the analysis grounded

Why the model only summarizes

Code collects the posts, and Claude only reads and summarizes them. Retrieval runs through the Reddit, Hacker News and DEV.to APIs, so I can see exactly which posts went in. If the model added what it already knew about a product, I couldn't trace those claims to a source, and that knowledge may be out of date or reflect press coverage more than user experience. My first version told the model to fill gaps with its general knowledge when data was thin. I removed that instruction, because it let the model produce findings that sounded grounded and weren't.

How I kept it grounded in the posts

- Each post sent to the model is numbered, and the prompt tells it to use only those posts.
- Every praise, complaint and theme must cite the post numbers that support it. The tool drops any item that cites nothing, or cites a post number that was never sent.
- Posts are sent whole, up to a 20,000-character limit, and the model is told how many were left out.
- When a product has too few posts, the model flags it, and the tool shows a warning and leaves the lists empty.
- If neither product returns any posts, the tool stops and asks for different search terms.

What it still gets wrong

- The citation check confirms that a cited post exists. It can't confirm the post supports the claim.
- The sentiment score, the percentages, the summary and the four competitive insights have no citation check, so the most strategic part of the output is the least verified.
- Reddit posts are added first, so they can fill most of the character limit and crowd out Hacker News and DEV.to.
- DEV.to is searched by tag using only the first word of the query. A search for "Microsoft Teams" looks at the "microsoft" tag and can return unrelated articles.
- All three sources lean toward technical, English-speaking users who post online. The results describe that group well and everyone else poorly.
- Two data problems went unnoticed for months. Hacker News returned no results from launch because of a search filter error, and Reddit requests from the live app failed until I moved them to Reddit's official API.

Telling a widespread signal from a loud anecdote

The tool doesn't make this distinction yet. One detailed complaint can become a pain point as easily as fifty short ones. Here's how I'd add it:

- Show how many separate posts support each item, using the citations the tool already collects.
- Treat an item as a pattern only when at least 3 posts from at least 2 sources support it, and label anything below that as a single report.
- Count posts from the same thread or author once, so one active user can't look like many.
- Avoid ranking by upvotes alone, since one viral post can carry thousands of votes.
- Rerun the same comparison over several weeks, and trust the themes that keep returning.

How I tested it

The engine pulls from three sources, Reddit across three subreddit scopes, Hacker News via Algolia, and DEV.to, aggregating roughly 100 posts per product per run. Each run produces a 10-field structured readout, sentiment score, a three way positive/neutral/negative breakdown, top praises, top complaints, top themes, and four competitive insights, in under a minute.

I ran the tool around 30 times in total; some were structured tests while others were exploratory.

Five PMs from my network separately ran their own comparisons and gave feedback on two things, did the insights feel plausible and concrete, and would they bring this into a stakeholder discussion. The response was mixed, some insights matched what they already suspected, others were genuinely new, and most leaned toward wanting to try it again rather than full confidence in using it as-is. Several noted that scaling this beyond a small test would need a broader source network.
